

Tecnológico de Monterrey

Campus Puebla

Etapa 2. Diseño

José Mariano Cariño Martínez	A01739206
Joseph Samberino Guerrero	A01739259
Esperanza Román Meléndez	A01739617
Leonardo Gómez Zamudio	A01736723

Integración de seguridad informática en redes y sistemas de software

Grupo 602

25/10/2026

Diagrama del diseño de la Base de datos

Se utilizará la infraestructura de Firebase Firestore para el diseño de la base de datos. Debido a las características de Zentemi, optamos por esta opción ya que:

Firestore incluye persistencia de datos local por defecto en SDKs móviles. El usuario puede completar una lección o mantener su racha sin conexión. Cuando la red vuelve, Firestore sincroniza automáticamente las métricas en segundo plano sin fallar ni mostrar errores de red.

Gracias a que la estructura a menudo toma forma de árbol con contenido educativo (Lecciones que contienen ejercicios que contienen recursos). La estructura de Firestore NoSQL permite un ajuste lógico que encaja con la naturaleza de la aplicación.

Firestore escala de manera totalmente automática sin necesidad de administrar servidores, balanceadores de carga o instancias. Soporta millones de conexiones concurrentes sin degradar la latencia.

A continuación se detalla un diagrama preliminar de colecciones de documentos que contienen los datos utilizados para las funciones de Zentemi.

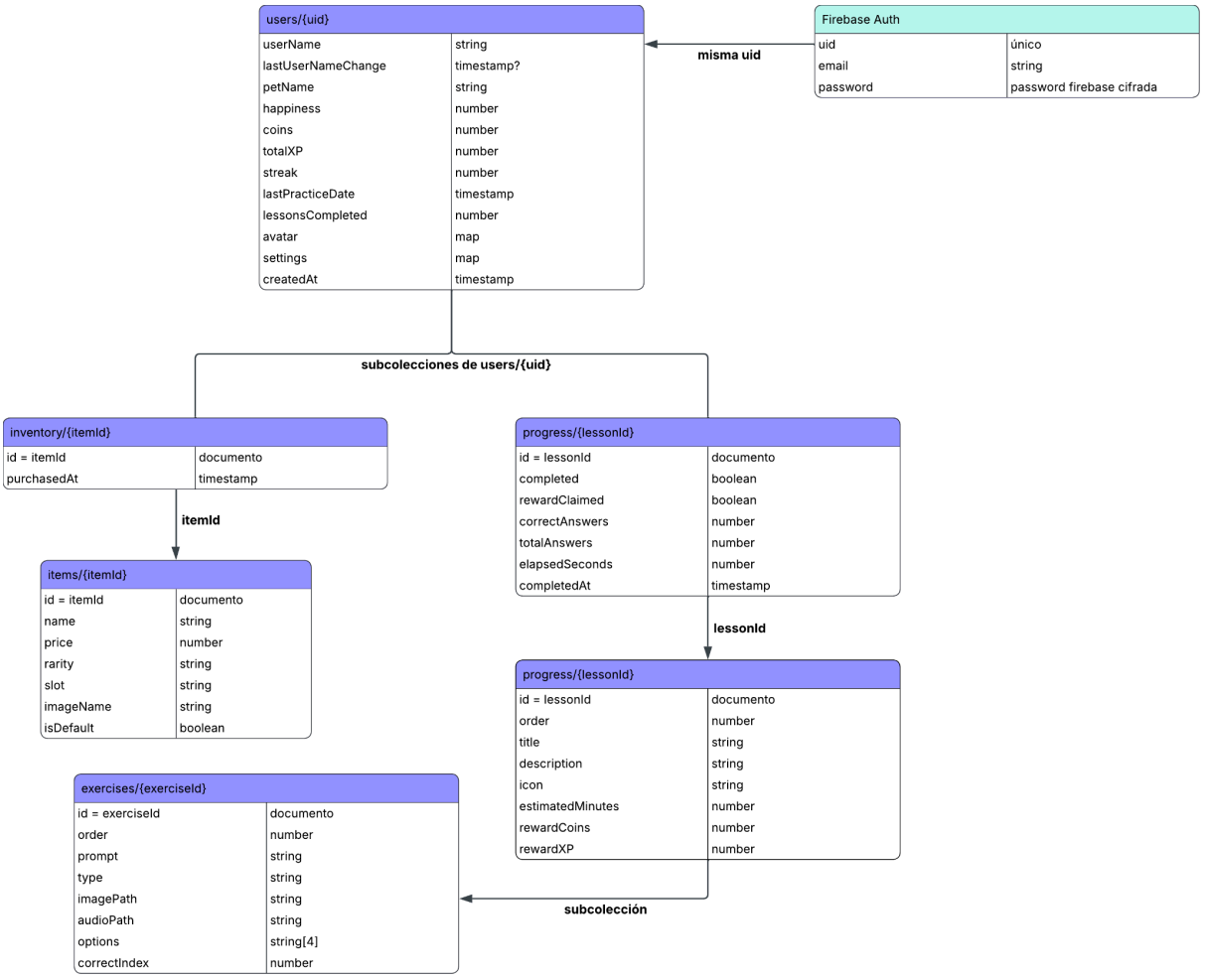


Diagrama de la arquitectura de la app

Diagrama de Componentes - Diagrama de Componentes

Se utilizará una arquitectura modular basada en el patrón MVVM (Model-View-ViewModel) en los clientes nativos (SwiftUI y Kotlin) y un backend Serverless con Google Cloud Functions. Debido a las características de Zentemi, optamos por este diseño ya que permite desacoplar la interfaz infantil (teclado fonético Yuhmu, mascota virtual y minijuegos) de la lógica de estado (AppState), facilitando que los equipos de UX/UI y Backend trabajen en paralelo manteniendo una experiencia fluida para niños de 6 a 15 años.

Asimismo, las operaciones críticas como el control de intentos de inicio de sesión con Circuit Breaker (checkLoginAttempts) y el hashing de contraseñas con scrypt (vía Firebase Auth), el registro con consentimiento parental (register), la evaluación de ejercicios (submitExercise) y la economía de monedas (purchaseItem) se procesan en el servidor para proteger la integridad educativa y evitar manipulaciones locales. Esta estructura optimiza los recursos del sistema al bloquear intentos de fuerza bruta antes de procesar la autenticación y aprovechar la caché local del SDK de Firebase para mantener respuestas menores a 300 ms.

A continuación se presenta el diagrama de componentes UML que modela los subsistemas e interfaces lógicas de Zentemi.

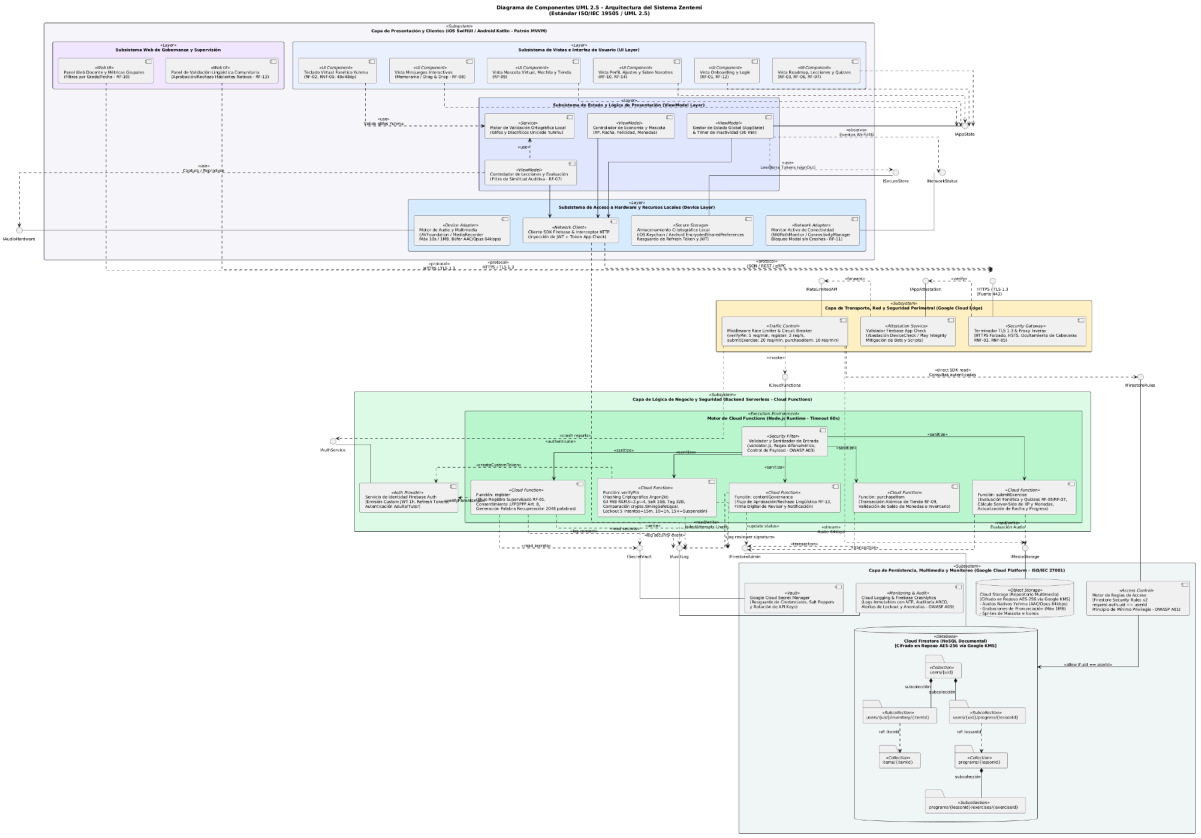
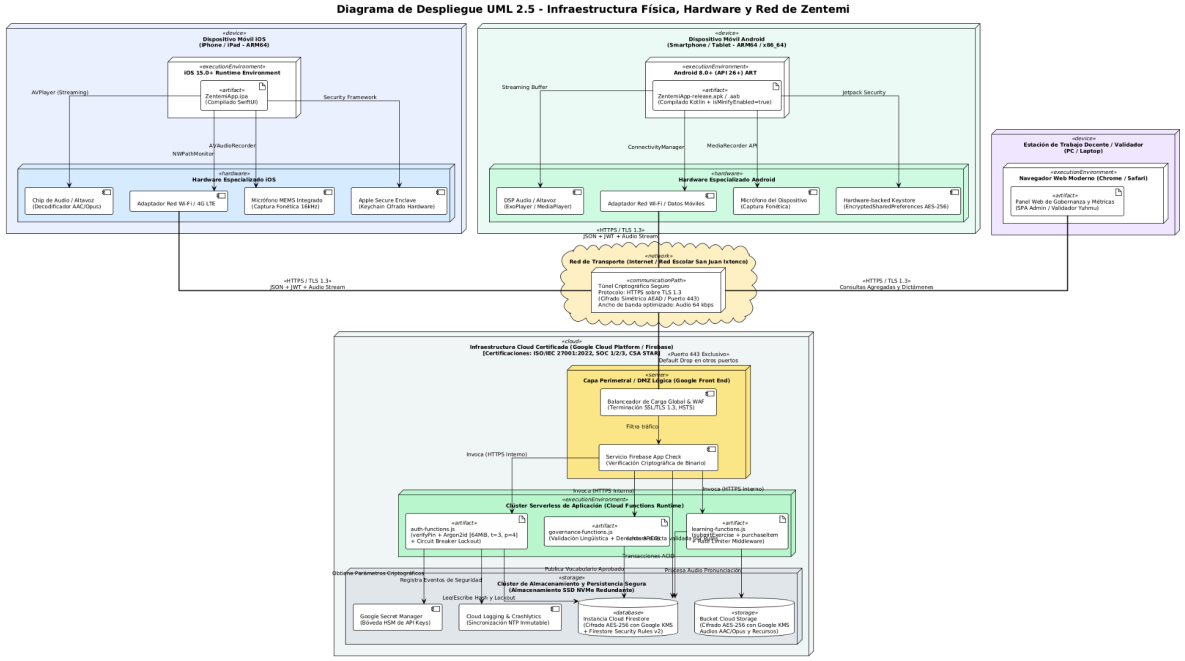


Diagrama de Despliegue - Diagrama de Despliegue.png

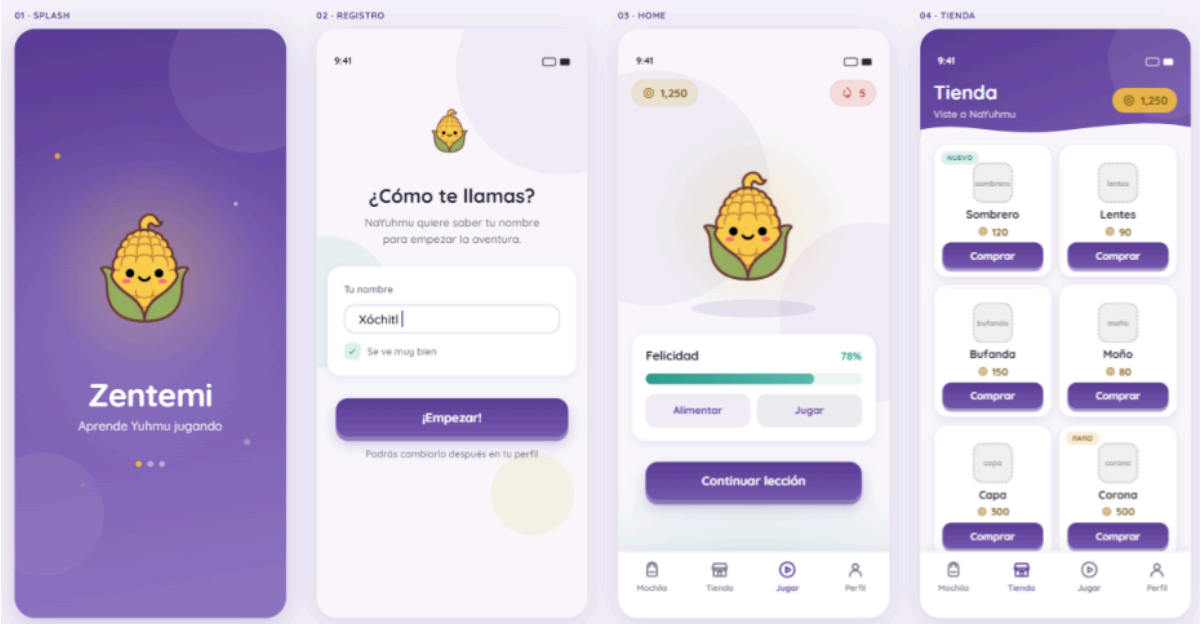
Para la infraestructura física, red y hardware especializado, se empleará un despliegue distribuido entre dispositivos móviles (iOS y Android) y la nube certificada de Google Cloud / Firebase (ISO/IEC 27001). Optamos por esta arquitectura ya que integra directamente los componentes de hardware del dispositivo necesarios para el aprendizaje del Yuhmu: el micrófono integrado para capturar ejercicios de pronunciación (máximo 10 s y 1 MB), el chip de audio para reproducir grabaciones de hablantes nativos y el almacenamiento seguro por hardware (Keychain en iOS y EncryptedSharedPreferences en Android) para resguardar los tokens de sesión.

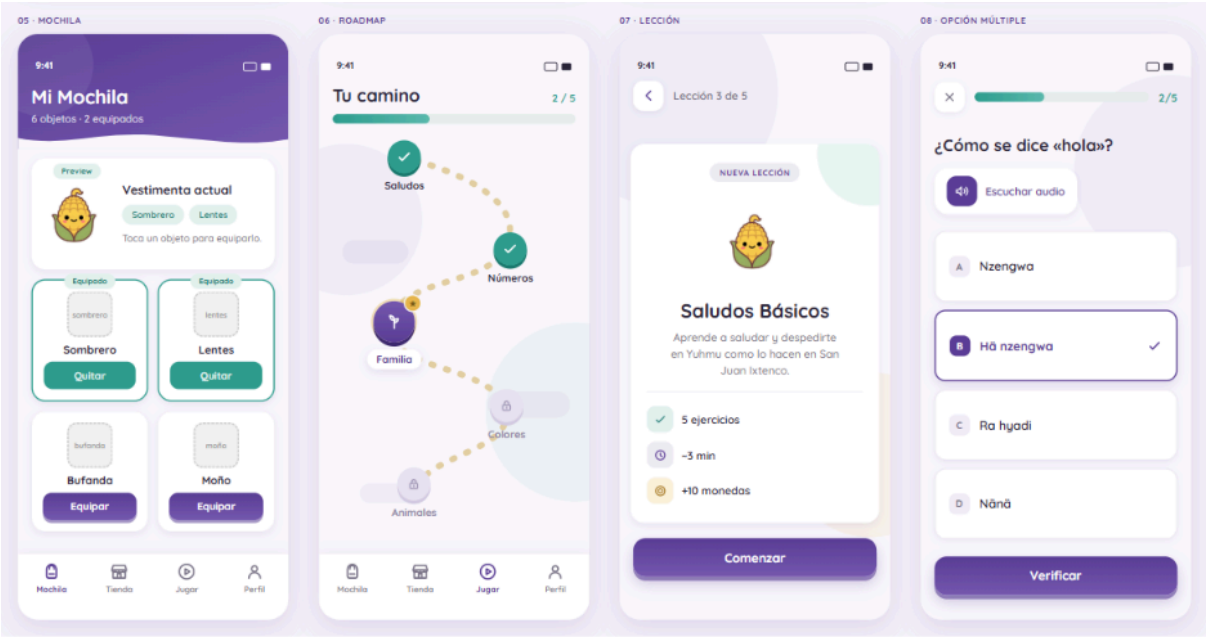
Además, para optimizar el consumo de red en la comunidad de San Juan Ixtenco, los audios en Cloud Storage se transmiten comprimidos en formato AAC/Opus a 64 kbps (logrando un inicio de reproducción menor a 1.5 segundos) mientras el adaptador de red monitorea la conectividad sin drenar la batería. Toda la comunicación viaja cifrada mediante HTTPS/TLS 1.3 validada por Firebase App Check, y los datos reposan cifrados con AES-256 mediante Google KMS.

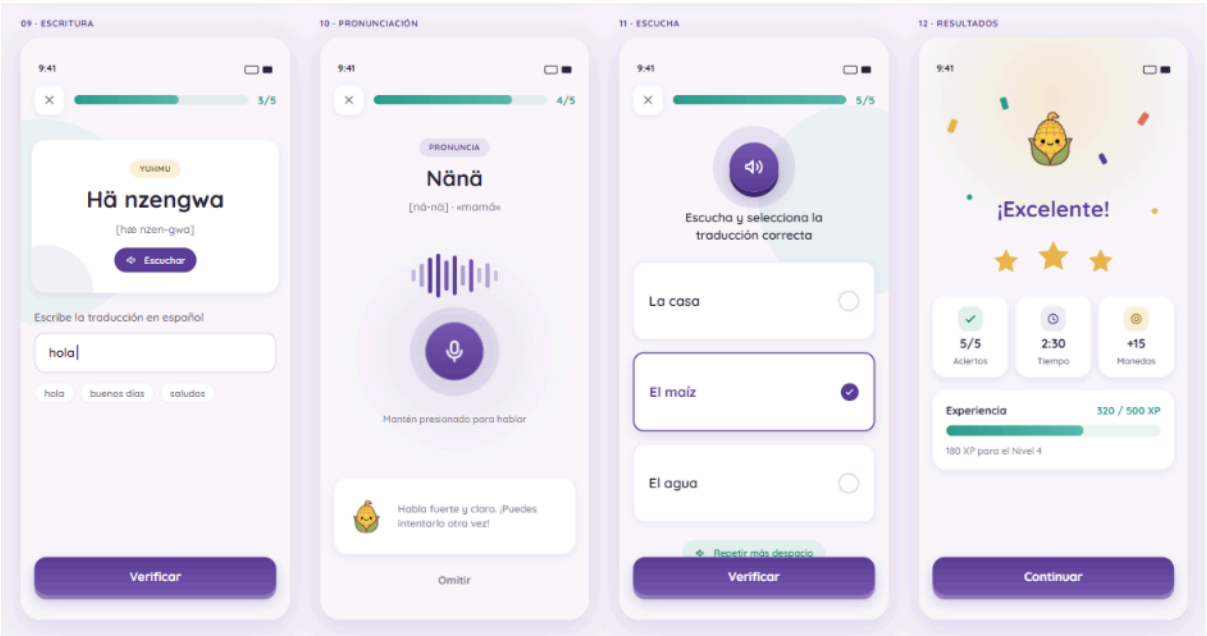
A continuación se detalla el diagrama de despliegue UML con los nodos de hardware, protocolos de red y servicios en la nube de Zentemi.

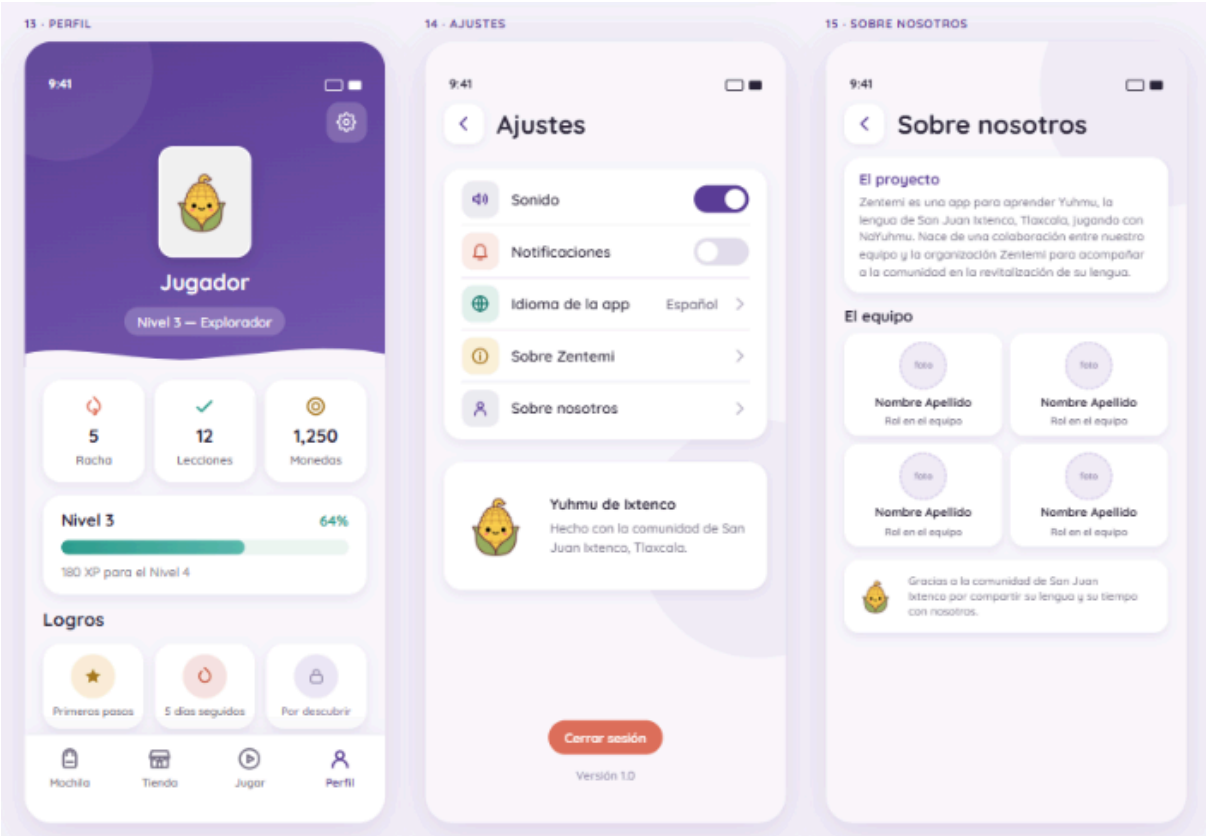


Diseño de las interfaces de usuario






09 · ESCRITURA
9:41
3/5
YUHMU
Hä nzengwa
[hæ nzen-gwa]
Escuchar
Escribe la traducción en español
hola
hola
buenos días
saludos
Verificar
10 · PRONUNCIACIÓN
9:41
4/5
PRONUNCIA
Nänä
[ná-nà] · «mamá»
Mantén presionado para hablar
Habla fuerte y claro. ¡Puedes intentarlo otra vez!
Omitir
11 · ESCUCHA
9:41
5/5
Escucha y selecciona la traducción correcta
La casa
El maíz
El agua
Repetir más despacio
Verificar
12 · RESULTADOS
9:41
¡Excelente!
5/5
Aciertos
2:30
Tiempo
+15
Monedas
Experiencia
320 / 500 XP
180 XP para el Nivel 4
Continuar


13 · PERFIL
9:41
Jugador
Nivel 3 — Explorador
5
Racha
12
Lecciones
1,250
Monedas
Nivel 3
64%
180 XP para el Nivel 4
Logros
Primeros pasos
5 días seguidos
Por descubrir
Mochila
Tienda
Jugar
Perfil
14 · AJUSTES
9:41
Ajustes
Sonido
Notificaciones
Idioma de la app
Español
Sobre Zentemi
Sobre nosotros
Yuhmu de Ixtenco
Hecho con la comunidad de San Juan Ixtenco, Tlaxcala.
Cerrar sesión
Versión 1.0
15 · SOBRE NOSOTROS
9:41
Sobre nosotros
El proyecto
Zentemi es una app para aprender Yuhmu, la lengua de San Juan Ixtenco, Tlaxcala, jugando con NdYuhmu. Nace de una colaboración entre nuestro equipo y la organización Zentemi para acompañar a la comunidad en la revitalización de su lengua.
El equipo
Nombre Apellido
Rol en el equipo
Nombre Apellido
Rol en el equipo
Nombre Apellido
Rol en el equipo
Nombre Apellido
Rol en el equipo
Gracias a la comunidad de San Juan Ixtenco por compartir su lengua y su tiempo con nosotros.

Integración de seguridad en sistemas computacionales

A continuación se definen las medidas técnicas que se integrarán al sistema para cumplir los requerimientos de seguridad del SRS y la normativa mexicana aplicable.

Resumen ejecutivo

Zentemi es una app móvil para niños de 6 a 15 años. Esto implica obligaciones legales reforzadas (LFPDPPP art. 8, LGDNNA art. 76) y una superficie de ataque que debe minimizarse desde el diseño.

Qué protegemos:

- Datos personales de menores: nombre de usuario, progreso, avatar, grabaciones de voz
- Integridad de la lógica educativa: ejercicios, evaluaciones, economía de monedas
- Disponibilidad del servicio para comunidades con conectividad limitada

Stack de seguridad elegido:

Capa	Tecnología	Propósito
Autenticación	Firebase Authentication	Gestión de identidad, tokens JWT
Base de datos	Cloud Firestore	Almacenamiento con cifrado en reposo automático (AES-256)
Hashing de credenciales	scrypt modificado (Firebase Auth)	Hash interno de contraseñas del adulto
Transporte	TLS 1.3 (forzado por Firebase)	Cifrado en tránsito
Reglas de acceso	Firestore Security Rules	Control de acceso por documento

Rate limiting	Firebase App Check + Cloud Functions	Protección contra abuso
Monitoreo	Firebase Crashlytics + Cloud Logging	Detección de anomalías

Autenticación

Modelo de acceso

El adulto (padre o tutor) es el único usuario que se autentica. El niño no tiene credenciales propias.

1. El adulto crea cuenta con email y contraseña vía Firebase Authentication
2. Acepta el aviso de privacidad con checkbox obligatorio antes de completar el registro (LFPDPPP art. 8, consentimiento expreso)
3. Crea uno o más perfiles de niño dentro de su cuenta (subcolección users/{uid})
4. Al abrir la app, inicia sesión y selecciona el perfil del niño
5. El niño usa la app dentro de la sesión del adulto

Cumple RF-01 (registro supervisado) y LGDNNA art. 76. Al eliminar el PIN de 4 dígitos se resuelven los hallazgos C-01 y C-02 de la auditoría (espacio de solo 10,000 combinaciones y palabra de recuperación hardcodeada).

Hashing de contraseña

Firebase Authentication hace el hash internamente con scrypt modificado (variante de Google). La contraseña nunca se almacena en texto plano y el equipo no necesita implementar hashing.

Recuperación de contraseña

Se usa el flujo nativo de Firebase Auth: el adulto pide un email de restablecimiento desde la pantalla de login. No hay palabra de recuperación custom, lo que elimina el hallazgo C-01 (palabra "AGUA" hardcodeada en el código).

Cifrado

En tránsito — TLS 1.3

Toda comunicación entre el cliente (iOS/Android) y Firebase se realiza sobre **TLS 1.3** por defecto. Firebase SDK fuerza HTTPS en todas las llamadas a Firestore, Authentication y Cloud Functions.

Qué viaja cifrado:

- Credenciales del adulto (email y contraseña sobre TLS)
- Tokens JWT de sesión
- Datos de progreso del niño
- Grabaciones de voz para ejercicios de pronunciación
- Respuestas de ejercicios

Cumple RNF-05 (TLS 1.3).

En reposo — AES-256

Cloud Firestore y Cloud Storage cifran automáticamente todos los datos en reposo con **AES-256** usando Google's Key Management Service (KMS). No requiere configuración adicional del equipo.

Capas de cifrado en reposo:

Dato	Dónde se almacena	Cifrado
Contraseña (hash)	Firestore (colección users)	AES-256 (KMS) + scrypt (Firebase Auth)
Nombre de usuario	Firestore	AES-256 (KMS)
Progreso y monedas	Firestore	AES-256 (KMS)
Grabaciones de voz	Cloud Storage	AES-256 (KMS)

Cumple RNF-04 (AES-256 en reposo).

Secretos y API keys

- GoogleService-Info.plist y google-services.json se añaden a .gitignore (nunca se commitean al repositorio)
- La API key actual (AIzaSyC5gxfGJSwIHr4xetkjHmIEOKIRH5KFGqc) se rota desde la consola de Firebase
- Se habilita **Firebase App Check** para que solo la app legítima pueda llamar al backend
- Variables sensibles de Cloud Functions se almacenan en **Secret Manager**, no en código

Protección contra inyecciones

Inyección SQL

Firestore es una base de datos NoSQL documental. **No usa SQL**, por lo tanto no es susceptible a inyección SQL clásica. Sin embargo, las consultas mal construidas pueden exponer datos no autorizados.

Mitigación:

- **Firestore Security Rules** restringen el acceso por documento: cada usuario solo puede leer/escribir sus propios datos
- Las consultas se construyen con el SDK tipado de Firebase (no se concatenan strings)
- Las Cloud Functions validan y sanitizan cada parámetro antes de ejecutar operaciones

Ejemplo de Security Rule:

```
None
1.  rules_version = '2';
2.  service cloud.firestore {
3.    match /databases/{database}/documents {
4.      match /users/{userId} {
5.        allow read, write: if request.auth != null
6.                           && request.auth.uid == userId;
7.      }
8.      match /users/{userId}/progress/{doc} {
9.        allow read, write: if request.auth != null
10.                          && request.auth.uid == userId;
11.     }
12.   }

  }
```

Inyección de JavaScript (XSS)

Zentemi es una app nativa (SwiftUI + Kotlin), no una app web. Esto elimina los vectores de XSS clásicos (DOM injection, reflected XSS). Sin embargo, se aplican medidas preventivas:

Validación de entrada en el cliente:

- Nombre de usuario: solo alfanuméricos y espacios, 3–20 caracteres, trim de whitespace
- Contraseña: mínimo 6 caracteres, validada por Firebase Auth
- Contenido generado por el usuario: sanitizado con expresiones regulares antes de enviarse

Validación en el servidor (Cloud Functions):

- Cada Cloud Function valida tipo, longitud y formato de sus parámetros
- Se rechazan payloads que excedan tamaños esperados
- Se usa la librería validator.js para sanitización de strings

Cumple OWASP A03:2021 (Injection) y resuelve hallazgo M-02 de la auditoría.

Circuit breaker, rate limiting y time limits

Circuit breaker en autenticación

El patrón circuit breaker protege contra ataques de fuerza bruta al login. Firebase Auth limita intentos del lado servidor, y este circuit breaker añade una capa adicional del lado aplicación.

Implementación (Cloud Function checkLoginAttempts):

Intentos fallidos	Acción	Duración del bloqueo
1–4	Se permite reintentar	—
5	**Lockout automático** (circuito abierto)	15 minutos
10 (acumulados)	Lockout extendido	1 hora

15+	Cuenta suspendida	Requiere verificación del adulto por email

Lógica:

1. Cada intento fallido incrementa failedAttempts y registra lastFailedAt en Firestore
2. Antes de procesar el login, la Cloud Function verifica si el usuario está en lockout
3. Si failedAttempts >= 5 y now - lastFailedAt < 15 min → rechazar sin procesar
4. El contador se resetea a 0 tras un login exitoso
5. Se envía notificación push al adulto cuando se activa un lockout

Cumple RNF-03 (protección contra fuerza bruta) y resuelve hallazgo M-01.

Rate limiting global

Firebase App Check verifica que las requests provengan de la app legítima (no de scripts o bots).

Límites por Cloud Function:

Endpoint	Límite	Ventana
checkLoginAttempts	5 req/min por IP	1 minuto
register	3 req/hora por IP	1 hora
submitExercise	20 req/min por usuario	1 minuto
purchaseItem	10 req/min por usuario	1 minuto

Se implementa con un middleware en Cloud Functions que usa un contador en Firestore (o Redis si se escala).

Cumple RNF-03 (rate limiting 20 req/s por IP del SRS).

Time limits (timeouts)

- **Tokens JWT de Firebase Auth:** expiran en 1 hora (configuración por defecto de Firebase)
- **Sesión inactiva:** si el niño no interactúa con la app por 30 minutos, se cierra la sesión automáticamente (el token no se refresca)
- **Cloud Functions:** timeout máximo de 60 segundos por invocación
- **Grabaciones de voz:** máximo 10 segundos por grabación, máximo 1 MB por archivo

Gestión de sesiones y logout seguro

Tokens de sesión

Firebase Authentication gestiona la sesión con un par de tokens:

- **ID Token (JWT):** expira en 1 hora. Se usa para autenticar cada request al backend
- **Refresh Token:** permite renovar el ID Token sin re-autenticarse. Se almacena en el Keychain (iOS) o EncryptedSharedPreferences (Android)

Logout real

El logout actual (auditoría hallazgo M-03) solo cambia una variable booleana. El logout correcto debe:

1. Llamar Auth.auth().signOut() (iOS) o FirebaseAuth.getInstance().signOut() (Android)
2. Esto invalida el ID Token y elimina el Refresh Token del dispositivo
3. Limpiar el estado local: AppState se reinicia a valores por defecto
4. Borrar cualquier dato cacheado en el dispositivo
5. Navegar a la pantalla de login

Sesión inactiva

Se implementa un timer que monitorea la inactividad del usuario:

- Cada interacción (tap, scroll, input) resetea el timer
- Si pasan 30 minutos sin interacción → se ejecuta el flujo de logout automático
- Se muestra una alerta 2 minutos antes del cierre: "¿Sigues ahí? Tu sesión se cerrará pronto"

Resuelve hallazgo M-03 (logout sin invalidación de sesión).

Cumplimiento legal

LFPDPPP (2025) — Datos personales de menores

Obligación	Artículo	Cómo se cumple

Consentimiento expreso para datos sensibles	Art. 8	Checkbox obligatorio al registrar cuenta del adulto
Aviso de privacidad	Arts. 14–17	Pantalla de aviso de privacidad antes del registro
Medidas de seguridad técnicas	Art. 18	scrypt (Firebase Auth), AES-256, TLS 1.3, Security Rules
Notificar vulneraciones	Art. 19	Logs + alerta automática al equipo + notificación al titular
Derechos ARCO	Arts. 21–34	Función de exportar/eliminar datos desde el perfil del adulto
Multas: 100–320,000 UMA (doble con datos sensibles)	Arts. 58–59	Todas las medidas anteriores minimizan el riesgo

LGDNNA — Niñez

- **Art. 76:** Protección de datos personales e intimidad de NNA → acceso restringido por Security Rules, minimización de datos (solo alias + edad + progreso)
- **Art. 101 Bis 3:** Acceso seguro a Internet para NNA → la app no expone al niño a contenido externo, no hay chat, no hay links a sitios web

OWASP Top 10:2021

Riesgo OWASP	Mitigación en Zentemi
A01 — Broken Access Control	Firestore Security Rules por usuario

A02 — Cryptographic Failures	AES-256 en reposo, TLS 1.3 en tránsito, scrypt para hashing
A03 — Injection	Validación client-side + server-side, SDK tipado
A04 — Insecure Design	Registro supervisado, circuit breaker, principio de mínimo privilegio
A07 — Identification and Authentication Failures	Firebase Auth + contraseña hasheada + lockout + tokens JWT
A09 — Security Logging and Monitoring Failures	Cloud Logging + Crashlytics + alertas

ISO/IEC 27001:2022

Firebase (Google Cloud) cuenta con certificación ISO 27001, SOC 1/2/3, y CSA STAR. La infraestructura cumple RNF-02 del SRS.

Checklist de implementación

Ordenado por prioridad. Cada item mapea a hallazgos de la auditoría y requerimientos del SRS.

Prioridad 1 — Antes de cualquier dato real (~2 semanas)

1. Configurar Firebase Authentication con email/contraseña para el adulto (elimina C-01, C-02)
2. Crear flujo de perfiles de niño: el adulto crea y selecciona un perfil dentro de su sesión (subcolección users/{uid}/children)
3. Escribir Firestore Security Rules que restrinjan acceso por auth.uid (elimina A-01, A-02, A-03)
4. Rotar la API key expuesta y añadir configs a .gitignore (elimina C-04, A-04)
5. Implementar flujo de registro supervisado con consentimiento parental (cumple RF-01)

Prioridad 2 — Antes de pruebas con usuarios (~1 semana)

1. Implementar circuit breaker en login (signInWithEmailAndPassword): lockout tras 5 intentos (elimina M-01, cumple RNF-03)
2. Implementar logout real con signOut() + limpieza de estado (elimina M-03)
3. Añadir validación de entrada en cliente y servidor (elimina M-02)
4. Habilitar Firebase App Check (elimina M-04)

Prioridad 3 — Antes de entrega final (~3 días)

1. Implementar timeout de sesión inactiva (30 min)
2. Mover lógica de monedas y ejercicios al servidor (elimina B-01, B-02)
3. Habilitar isMinifyEnabled = true en Android release
4. Configurar Cloud Logging con alertas para eventos de seguridad
5. Redactar aviso de privacidad y pantalla de derechos ARCO

Fuentes

Aspectos básicos. (n.d.). Firebase Documentation. https://firebase.google.com/docs/guides?hl=es-419

Comienza a usar la edición Standard de Firestore | Firebase. (n.d.). Firebase. https://firebase.google.com/docs/firestore/quickstart?hl=es-419#kotlin_1

Protocol, O. (2026, October 23). *Offline Protocol vs Firebase offline persistence*. Offline Protocol. https://www.offlineprotocol.com/learning/offline-first/offline-protocol-vs-firebase-offline-persistence